



JAVA using BlueJ



Elementary Concept of OOPS

Unit 1

Object Oriented Programming

- Object-oriented Programming System(OOPs) is a programming technique that is based on the **Class and Object** Model.
- In OPPS first we create layout(blue print) then implement (create) it.
- The primary goal of object-oriented programming is to increase the **flexibility** and **maintainability** of programs.
- OOPs is enable to solve the **real life problem** through program.



- A class can be defined as a template/blueprint that describes the behavior/state that the object of its type support.
- It is also known as *User defined data Type*.
- It is only a **logical component** and not the physical entity.

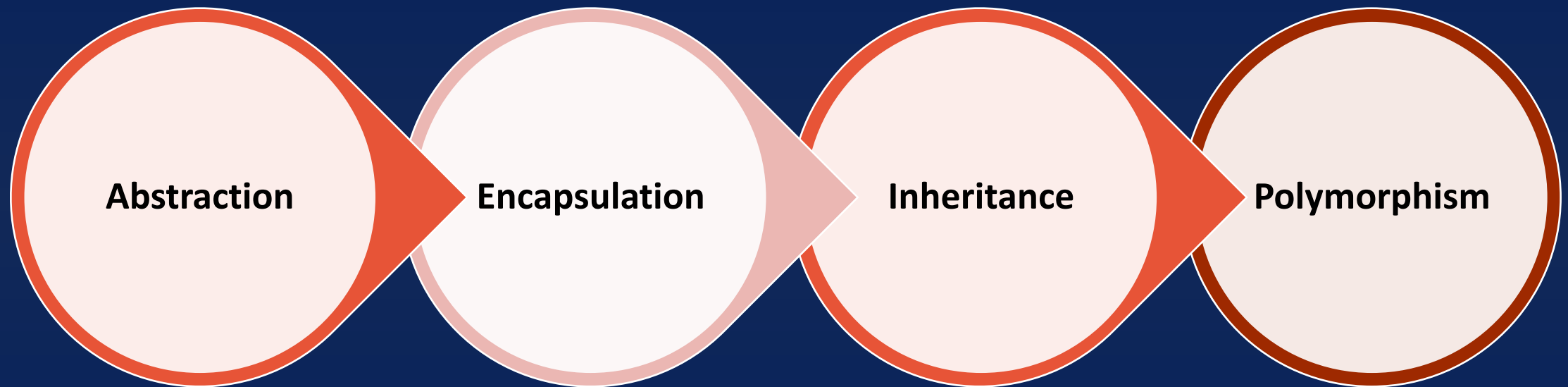


Object

- An object can be defined as an **instance of a class**, and there can be multiple instances of a class in a program.
- Objects have states and behaviors.
- It has real life (physical) existence.



Principals of OOPs



1. Abstraction

- Abstraction refers to showing only the essential features of the application and hiding the details.
- In java, classes provide methods to the outside world to access & use the data variables, but the variables are hidden from direct access.
- This can be done by access specifiers.



Encapsulation

- Encapsulation is all about binding the data variables and functions together in class.
- It can also known as **Data Binding**.



- Inheritance is a way to reuse once written code again and again.
- The class which is inherited is called base class & the class which inherits is called derived class. So when, a derived class inherits a base class, the derived class can use all the functions which are defined in base class, hence making code reusable.



- It is a feature, which lets us create functions with same name but different arguments, which will perform differently.
- That is function with same name, functioning in different way. Or, it also allows us to redefine a function to provide its new definition.



Advantages of OOPs

- OOPs Concepts in Java offer easy to understand and a clear modular structure for programs.
- OOPs is faster and easier to execute
- OOPs helps to keep the Java code DRY "Don't Repeat Yourself", and makes the code easier to maintain, modify and debug
- OOPs makes it possible to create full reusable applications with less code and shorter development time



Introduction Java

Introduction

- Java is a general-purpose, robust, class-based , object-oriented language that is specifically designed to build platform independent applications. It Facilitates the application developers with the feature of “Write Once run Anywhere”.
- Java is a third generation programming language originally developed by James gosling at sun microsystems. Java was released in 1995 as a core component of Sun Microsystems java platform.



Types of Java Program

- Java is used to create two types of a application. These are descried as follows:
- **Stand Alone Application:** Stand alone refers to those java program that run independently on a computer without requiring external support. Stand alone application can either be graphical user interface(GUI), or console based.
 - **Console based Application:** these application run through a command prompt, and do not have GUI based screen
 - **GUI Based Application :** These application have Graphical user interface like button ,textbox, etc.
- **Internet Applets :** Applets are small java application, mainly use for internet programming



Java Compilation Process

- An ordinary compiler simply convert the source code (written by a user) into a machine code.
- A machine code is dependent on a specific platform, programming environment or a machine. Hence, it cannot run/execute on every machine such a code is called the **native code**.
- Unlike other compiler , the java compiler converts the source code into a format, which is neither dependent on a machine nor a platform.
- We call this new code as the Java **byte Code**. Further the byte code is converted into an executable machine code for a particular platform by **JVM (java Virtual machine)** using the corresponding interpreter.



- Like other compilers, java compiler does not create the machine code for any particular platform.
- It produces an **intermediate code** in a special format, which can be interpreted for different platforms.
- This special format is known as the **byte code**.
- The same byte code is used by all interpreters to create various type of machine code supporting corresponding platforms.



The Java Virtual Machine (JVM)

- The code written by using any programming language, is either compiled or interpreted before execution. But in Java, we perform both operations.
- First of all, we compile the code into byte code. Further, this byte code is interpreted at the platform level, on which the application is to be executed.
- For every platform, we have a special java interpreter, which interprets the java byte code.
- The devices (computer, mobile, etc.), which are equipped with interpreter that can read the byte code , are called **Java Virtual Machines**.
- Hence any software that has the interpreter to read the **java byte code** can be referred to as a **java virtual Machine**.



Characteristics of Java

- **Platform Independent** : the concept of “write once, run anywhere”, is one of the most important features of java, which makes it a very powerful language. Java applications can run on almost every platform, provided it should have the Java Runtime Environment (JRE)
- **Object Oriented** : Object-Oriented programming models the real world in term of object. A java programs is object oriented because programming in java is centered on creating objects, manipulating objects, and making objects work together. It provides great flexibility, clarity and reusability through Encapsulation, Polymorphism and Inheritance.
- **Robust** : java has a strong memory management system. It has a unique compilation system, in which the compiler checks the program errors and the interpreter checks any run time errors, and makes the system secure. This two step compilation makes java a robust language.



Characteristics of Java (cont..)

- **Distributed** : Distributed computing involves several computers working together on a network. The widely used protocols like HTTP and FTP are developed in java.
- **Secure** : java uses an encryption system to allow the Java applications to transmit data over the internet in an encrypted format. In this manner; It makes data transmission on secure, using the java applications.
- **Multithread** : Java is also a multithreaded programming language. Multithreading works in similar way as multiple processes run on one computer.
- **Support multimedia** : Java application support multimedia content, i.e., graphics, animation, image and video.
- **Freely Available** : It can be downloaded and distributed freely.



Values and Types

The Character set is the defined list of characters, recognized by the compiler or interpreter.

Letters	Digits	White Space
Capital A to Z	All decimal digits 0 to 9	Blank Space
Small a to z		Horizontal tab
		Vertical tab
		New Line



Special Character

,	Comma	&	Ampersand
.	Dot	^	Caret
;	Semicolon	*	Asterisk
:	Colon	-	Minus
'	Apostrophe	+	Plus
"	Quotation Mark	<	Less than
!	Exclamation mark	>	Greater than
	Vertical bar	()	Parenthesis
/	Slash	[]	Bracket
~	Tilde	%	Percent
_	Underscore	#	Hash
\$	Dollar	=	Equal to
?	Question mark	@	At the rate

Tokens

java tokens are the basic building blocks in java language which are constructed together to write a java program. Each and every smallest individual units in a java program are known as java tokens.

- Punctuators / Separators
- Keywords
- Identifiers
- Constants
- Variable
- Data Types
- Operators



Punctuators / Separators

Every language has a unique set of special symbols that work as punctuators / separators. In java, punctuators are used to grouping and separating the data values written in java programs.

Symbol	Name	Purpose
;	Semicolon	Terminate the statement
()	parenthesis	Used in expression and function
[]	Square bracket	Use for array declaration
{ }	Curly Brace	Scope of the statement
,	Comma	Variable separator
.	Period (dot)	to access the member variables and methods.

Keywords are special words that have special meaning in programming language. They represent a specific task to be performed. Keywords cannot be used as identifiers. All keywords are used in lowercase.

<code>abstract</code>	<code>do</code>	<code>implements</code>	<code>private</code>	<code>throw</code>
<code>boolean</code>	<code>double</code>	<code>import</code>	<code>protected</code>	<code>throws</code>
<code>break</code>	<code>else</code>	<code>instanceof</code>	<code>public</code>	<code>transient</code>
<code>byte</code>	<code>extends</code>	<code>int</code>	<code>return</code>	<code>true</code>
<code>case</code>	<code>false</code>	<code>interface</code>	<code>short</code>	<code>try</code>
<code>catch</code>	<code>final</code>	<code>long</code>	<code>static</code>	<code>void</code>
<code>char</code>	<code>finally</code>	<code>native</code>	<code>super</code>	<code>volatile</code>
<code>class</code>	<code>float</code>	<code>new</code>	<code>switch</code>	<code>while</code>
<code>continue</code>	<code>for</code>	<code>null</code>	<code>synchronized</code>	
<code>default</code>	<code>if</code>	<code>package</code>	<code>this</code>	

Identifiers

All Java components require names. Names used for classes, variables, and methods are called identifiers.

- It can be only combination of alphabet (a-z, A-Z), digits (0-9) , under score) (_) and (dollar (\$) sign.
- All identifiers should begin with a letter (A to Z or a to z), dollar (\$) or an underscore (_).
- A keyword cannot be used as an identifier.
- identifiers are case sensitive and have no maximum length.



Data Type

Data type is a special keyword used to allocate sufficient memory space for the data, in other words Data type is used for representing the data in main memory (RAM) of the computer.

In general every programming language is containing three categories of data types.
They are

- Fundamental or primitive data types
- Derived data types
- User defined data types.



Primary Data Type

Type	Size	Range	Default Value
byte	1 byte	-128 to 127	0
short	2 bytes	-32,768 to 32,767	0
int	4 bytes	-2,147,483,648 to 2,147,483, 647	0
long	8 bytes	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	0
float	4 bytes	3.4e-038 to 3.4e+038	0.0f
double	8 bytes	1.7e-308 to 1.7e+038	0.0d
char	2 byte	0 to 65,536 (unsigned)	'\u0000'
boolean	1 byte	true or false	false

Variable

It is a data name used for storing a data value. Its value may be changed during the program execution. The value of variables keeps on changing during the execution of a program.

Declaration of Variable:

<data type> <variable name>

Example;

```
int num;
```

Initialization on variable:

<data type> <variable name>= <value>;

Example:

```
int num=34;  
char gen='m';  
float amt=444.5f;
```



Values do not change during the execution of the program

Numerical constants:

- **Integer constants:** These are the sequence of numbers from 0 to 9 without decimal points or fractional part or any other symbols.

Eg: 10, 20, + 30, - 14

- **Real constants:** It is also known as floating point constants.

Eg: 2.5, 5.342

Character constants:

- **Single character constants:** A character constant is a single character. Characters are also represented with a single digit or a single special symbol or white space enclosed within a pair of single quote marks.

Eg: 'a', '8', " ".

- **String constants:** String constants are sequence of characters enclosed within double quote marks.

Eg: "Hello", "india", "444"



Operators

An operator is a symbol that tells the compiler to perform specific mathematical or logical functions. C language is rich in built-in operators and provides the following types of operators :

- Arithmetic Operators
- Relational Operators
- Logical Operators
- Bitwise Operators
- Assignment Operators
- Increment and Decrement Operators
- Conditional Operators
- Special Operators



An arithmetic operator performs mathematical operations such as addition, subtraction and multiplication on numerical values (constants and variables).

Operator	Description	Example (int a=10,b=7;)
+	Addition	$a+b = 17$
-	Subtraction	$a-b = 3$
*	Multiplication	$a*b = 70$
/	Division	$a/b = 1$
%	Modulus (Remainder)	$a\%b = 3$
Note: modulus operator only used with int data type		

A relational operator checks the relationship between two operands. If the relation is true, it returns 1; if the relation is false, it returns value 0.

Relational operators are used in decision making and loops.

Operator	Description	Example (int a=5,b=10)
>	greater than	$a > b = 0$
<	less than	$a < b = 1$
>=	greater than or equal to	$a >= b = 0$
<=	less than or equal to	$a <= b = 1$
==	equal to	$a == b = 0$
!=	not equal to	$a != b = 1$

Logical operators are used to combine two or more condition for building complex expression.

Operator	Description	Example
&&	Logical AND. True only if all operands are true	If a = 1 and b = 0 then (a && b) is False
	Logical OR. True only if either one operand is true	If a = 1 and b = 0 then (a b) is True
!	Logical NOT. True only if the operand is 0	If a = 1 then !(a) is False

Bitwise operators perform manipulations of data at bit level. These operators also perform shifting of bits from right to left. Bitwise operators are not applied to float or double.

Operator	Description	Example: a=0,b=1
&	Bitwise AND	a&b =0
	Bitwise OR	a b =1
^	Bitwise exclusive OR	a^b =1
<<	Left Shift	
>>	Right Shift	

EXAMPLE: Bitwise Operators in C

a = 0001000

b= 2

a << b = 0100000

a >> b = 0000010



- Increment operators are used to increase the value of the variable by one and decrement operators are used to decrease the value of the variable by one in C programs.

Type of Increment Operator

- Pre increment/decrement Operator (++a, --a)
- Post increment/decrement Operator (a++, a--)
- In pre increment / decrement first increment/decrement the value of variable and then used inside the expression (initialize into another variable)
- In post-increment / decrement first value of variable is used in the expression (initialize into another variable) and then increment / decrement the value of variable.



Conditional Operator

The conditional operator (? :) is a ternary operator. This operator needs 3 operands to perform the operation.

Syntax:

Variable = <test expression> ? <expression 1> : <expression 2>

The conditional operator works as follows:

- Test Expression is Condition
- Expression 1 is Statement Followed if Condition is True
- Expression 2 is Statement Followed if Condition is False



Precedence and Associativity of operators

- **Precedence:** If more than one operators are involved in an expression, C language has a predefined rule of priority for the operators. This rule of priority of operators is called operator precedence.
- **Associativity:** If two operators of same precedence (priority) is present in an expression, Associativity of operators indicate the order in which they execute.



Operator	Description	Associativity 39
() [] . -> ++ --	Parentheses (function call) (see Note 1) Brackets (array subscript) Member selection via object name Member selection via pointer Postfix increment/decrement (see Note 2)	left-to-right
++ -- + - ! ~ (type) * & sizeof	Prefix increment/decrement Unary plus/minus Logical negation/bitwise complement Cast (convert value to temporary value of <i>type</i>) Dereference Address (of operand) Determine size in bytes on this implementation	right-to-left
* / %	Multiplication/division/modulus	left-to-right
+ -	Addition/subtraction	left-to-right
<< >>	Bitwise shift left, Bitwise shift right	left-to-right
< <= > >=	Relational less than/less than or equal to Relational greater than/greater than or equal to	left-to-right
== !=	Relational is equal to/is not equal to	left-to-right
&	Bitwise AND	left-to-right
^	Bitwise exclusive OR	left-to-right
	Bitwise inclusive OR	left-to-right
&&	Logical AND	left-to-right
	Logical OR	left-to-right
? :	Ternary conditional	right-to-left
= += -= *= /= %= &= ^= = <<= >>=	Assignment Addition/subtraction assignment Multiplication/division assignment Modulus/bitwise AND assignment Bitwise exclusive/inclusive OR assignment Bitwise shift left/right assignment	right-to-left
,	Comma (separate expressions)	left-to-right

Type Casting

Type casting is process to convert a variable from one data type to another data type. For example if we want to store a integer value in a float variable then we need to typecast integer into float.

There are tow type of conversion

Implicit: When the type conversion is performed automatically by the compiler without programmers intervention, such type of conversion is known as **implicit type conversion**

Explicit: The type conversion performed by the programmer by posing the data type of the expression of specific type is known as explicit type conversion. The explicit type conversion is also known as **type casting**.

Syntax:

```
(data_type) var_name;
```

Example:

```
int =65;  char ch=(char)a;
```



Comments

- Three permissible styles of comment in a Java technology program are:
 - `//` comment on one line
 - `/*` comment on one or more lines `*/`
 - `/**` documenting comment `*/`



Flow Control Statement

The statements inside your source files are generally executed from top to bottom, in the order that they appear. *Control flow statements*, however, break up the flow of execution by employing decision making, looping, and branching, enabling your program to *conditionally* execute particular blocks of code.

- 1 • Conditional Statement
- 2 • Looping statement
- 3 • Jumping Statement



Conditional statement

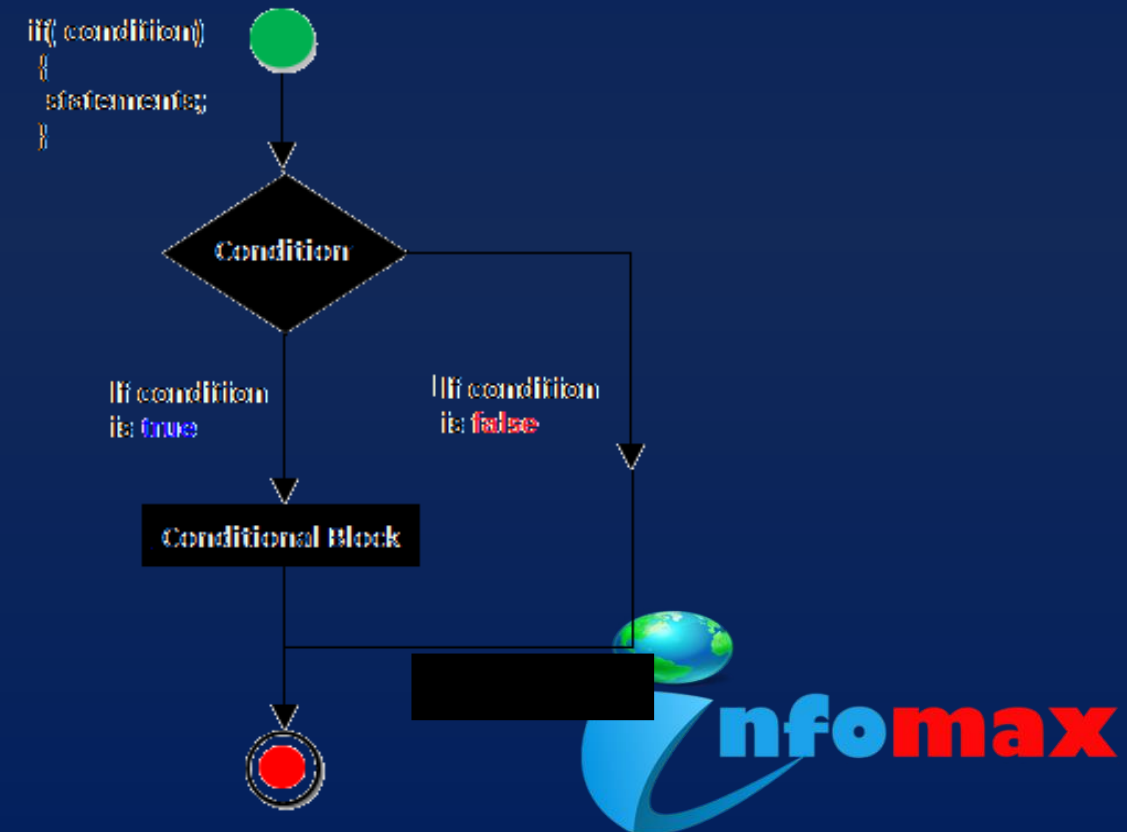
- **Decision making statement** is depending on the condition block need to be executed or not which is decided by condition. In java programming language there are three types of conditional statement:
 - If statement
 - If else statement
 - Nested if else statement
 - Else if ladder statement
 - Switch case statement



if Statement

- This is the simplest form of branching statement. When we have only one condition to evaluate, we should use this statement. It performs a course of action if the condition evaluates to true, otherwise, it ignores the action.
- **Syntax:**

```
If(conditional expression)
{
    Statement 1
    Statement 2
    .....
    Statement n
}
```



- Write a program to check whether the entered number is positive or not.
- Write a program to input an integer number and print its absolute value.
- Write a program input any char from user and convert it in upper case if it is in lower case.



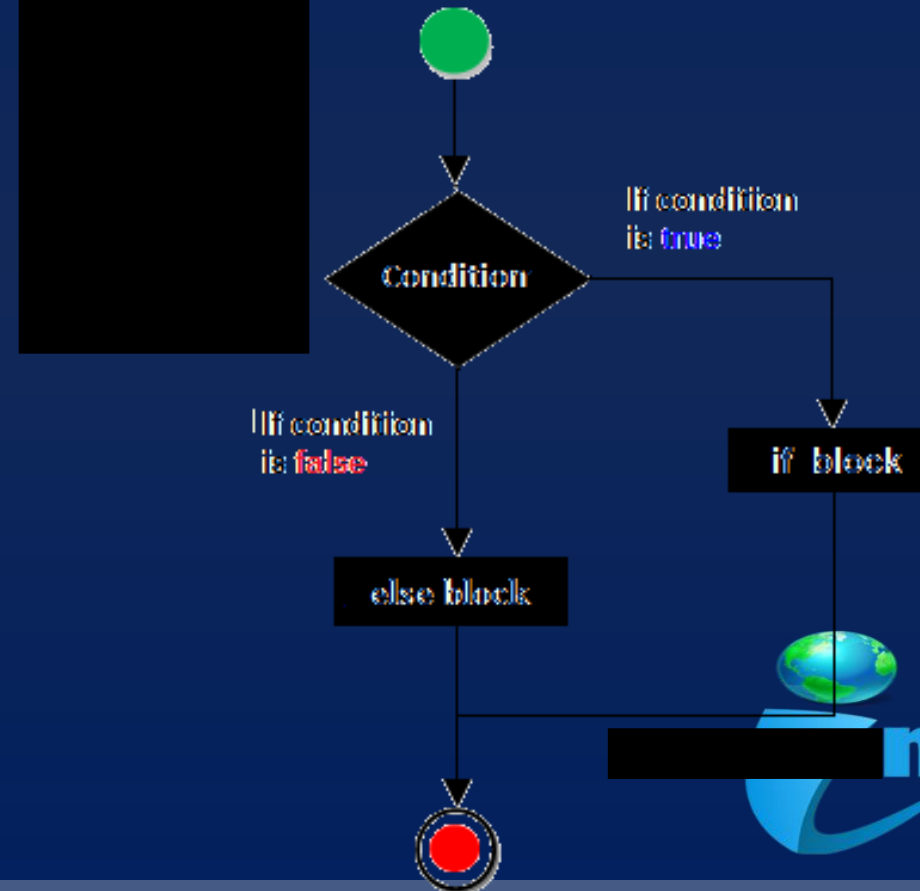
if-else statement

The if else control structure is used where either of two action are to be performed depending upon the result of a conditional expression. It contains two block of statement. If the conditional expression evaluates to true, the statement(s) in if block are executed and if the result is false, then statement(s) in the else block get executed.

Syntax:

```

if(conditional expression)
{
    statement 1
    statement 2
    .....
}
else
{
    statement 1
    statement 2
    .....
}
  
```

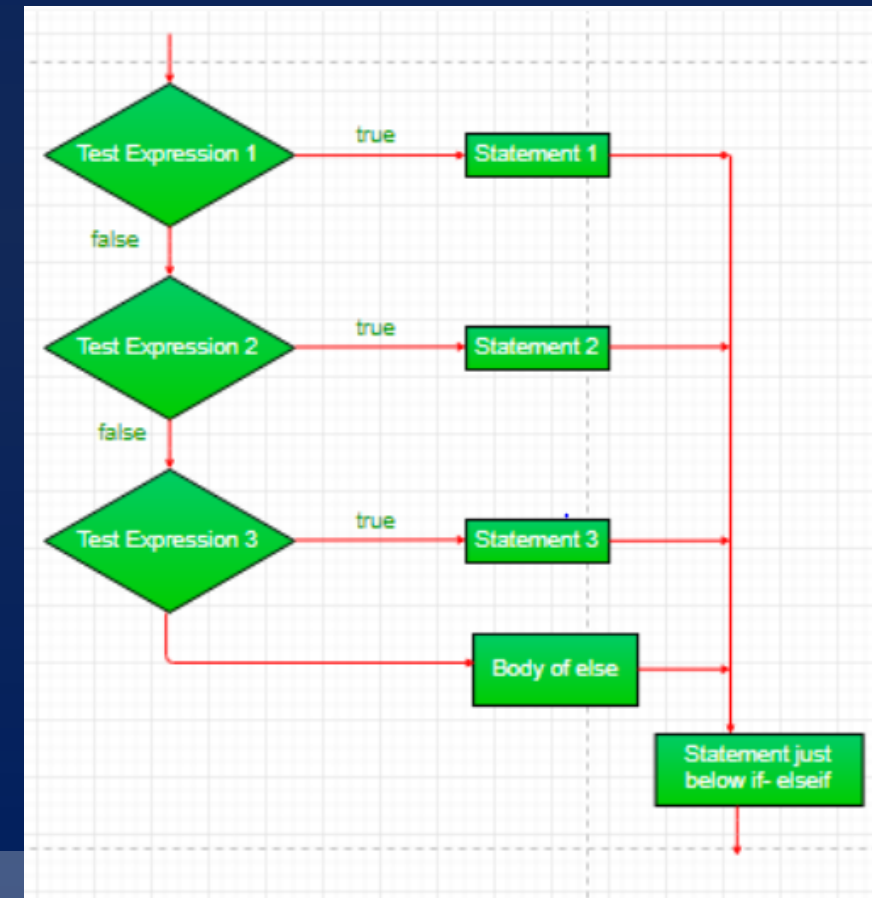


- Write a program to input an integer number and check number is **EVEN** or **ODD**.
- Write a program to input any integer number and check number is **BUZZ Number** or Not. (A number that ends with 7 or divisible by 7, is called buzz number).
- Write a program to display whether the entered character is Uppercase or Lowercase.
- Write a program to check whether the entered number is divisible by 5 or Not.
- Write a program input cost price, selling price and check whether is profit or loss and also print profit/loss amount.
- Write a program input any one digit integer number and its square contains as last digit or not. Ex: 5=25 , 6=36



- The if-else-if ladder statement executes one condition from multiple statements. The execution starts from top and checked for each if condition. The statement of if block will be executed which evaluates to be true. If none of the if condition evaluates to be true then the last else block is evaluated.

```
if(condition_expression_One)
{
    statement1;
}
else if (condition_expression_Two)
{
    statement2;
}
else if (condition_expression_Three)
{
    statement3;
}
else
{
    statement4;
}
```



- Write a program to input serial number and print the corresponding day of week.
- Write a program input 5 subject marks and find total, percentage and grade based on given condition.
 - Grade A – per ≥ 80
 - Grade B – per ≥ 70 and < 80
 - Grade C – Per ≥ 60 and < 70
 - Grade D – per ≥ 50 and < 60
 - per < 50 is FAIL
- Create a program to calculate the amount that a customer pays for the taxi that he hires based on the following condition

KMS Travelled	Amount per KM(Rs)
First 10 Kms	30
Next 20 Kms	20
Next 40 Kms	15
Above 70 Kms	12



- Nested if is a conditional statement, which is used when more than one condition is required to be checked at the same time in a program. The statements are executed from the top to the bottom, checking each condition, whether it meets the criteria or not. If the condition is true, then it executes the block of associated statements of true part, else it goes to the next condition to execute.
- Nesting an if means you can use one if or else if statement inside another if or else if statement. The inner if(s) must always be terminated before the outer if(s).



- Program that will accept three numbers from the user , and find the largest number.

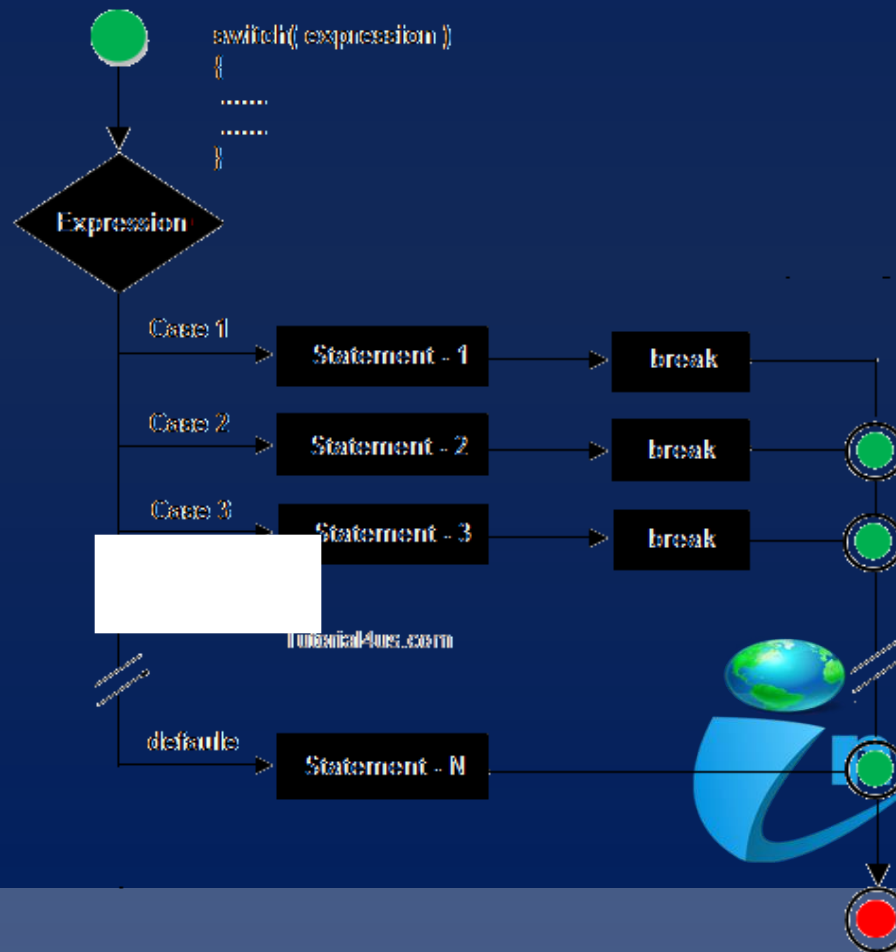


Switch Statement

A switch statement is used when there is requirement to check multiple condition in a program. It provides an easy way to branch to different parts of the code in program based on the value of an expression. It is a substitute for the large series of else if statements.

Syntax:

```
switch(expression)
{
    case constant:
        statement;
        break;
    case constant:
        statement;
        break;
    ..
    ..
    default:
        statement;
}
```



Looping statement

- A for loop is used to repeat a block of statement enclosed in curly braces. It contains three optional expressions enclosed in parentheses and separated by semicolons, followed by statements executed in the loop.
- For loop
- While loop
- Do while loop



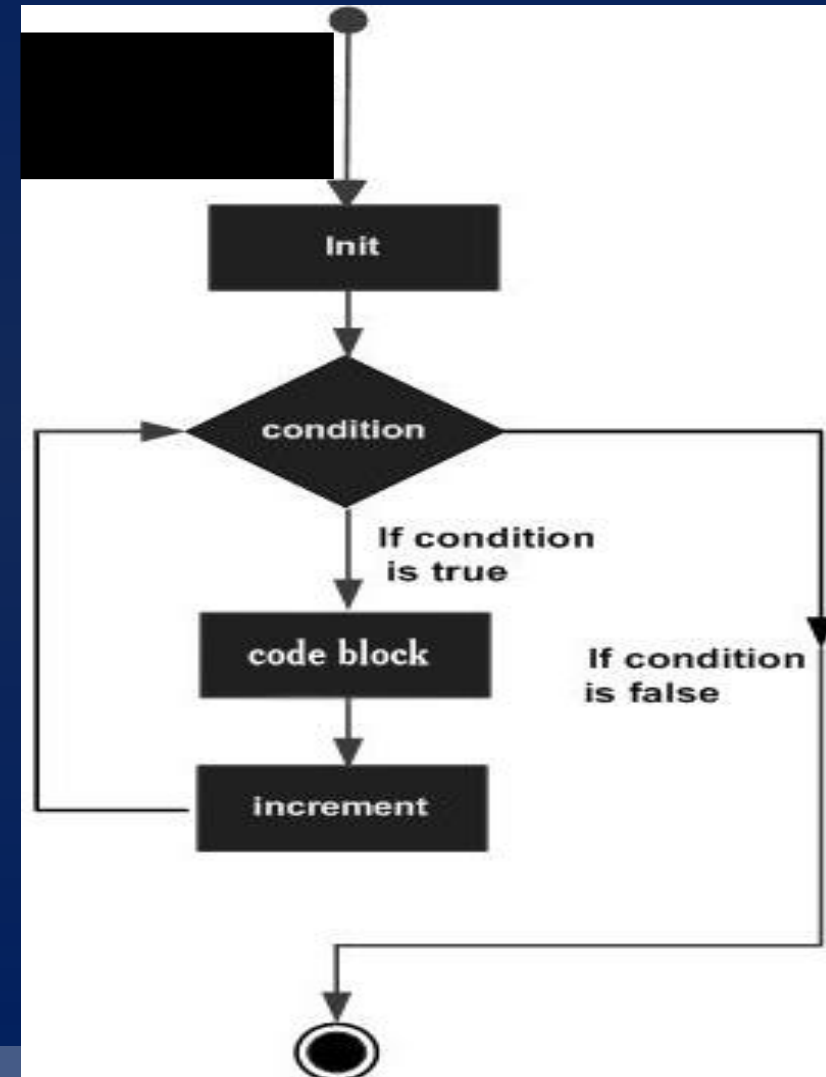
For Loop

for loop is a statement which allows code to be repeatedly executed. For loop contains 3 parts Initialization, Condition and Increment or Decrements.

Syntax:

```
for(initial value ;condition;  
increment/decrement)
```

```
{  
    statement 1  
    statement 1  
    .....  
    statement n  
}
```



- A for loop can have multiple variations that makes more flexible and efficient.
 - Multiple initialization and update expression
 - Optional expression
 - Infinite loop
 - Empty loop



- A for loop may contain multiple initialization and update expressions. The multiple expressions must be separated by commas.

Example:

```
int i,j;
```

```
for(i=1,j=20; i,<=10; i++,j--)
```

```
{
```

```
    System.out.println(i+" "+j)
```

```
}
```

Multiple Initialization

Multiple update expression



- By now you are familiar that a for loop contains three types of expressions: initialization expression(condition) and an update expression. But, all these expressions are optional, i.e., we can skip any or all of these expressions depending upon the requirement of the program.

Case 1:

```
Int i=2;  
for( ;i<=20;i=i+2)  
{  
    System.out.println(i);  
}
```

Case 2:

```
Int i=2;  
for( ;i<=20; )  
{  
    System.out.println(i);  
    i=i+2;  
}
```

Case 3:

```
Int i=2;  
for( ; ; )  
{  
    System.out.println(i);  
    i=i+2;  
    if(i>20)  
    {  
        break;  
    }  
}
```



- A for loop will execute its statement block infinite number of times if we skip the condition. If the test expression (condition) is not present in the loop, it will never return false. Hence, it will execute endlessly and become an infinite loop.

Case 1

```
int l  
for(i=1; ;i++)  
{  
    System.out.println(i);  
}
```

Case 2

```
int i;  
for( ; ; )  
{  
    System.out.println(i);  
}
```

Case 3

```
int i;  
for(i=1;i<=i;i++)  
{  
    System.out.println(i);  
}
```

Here , $i \leq i$ will always be true.



➤ A loop without any statement in its body is called an empty loop

```
int i
for(i=2;i<=50;i++)
{
}
```

Or

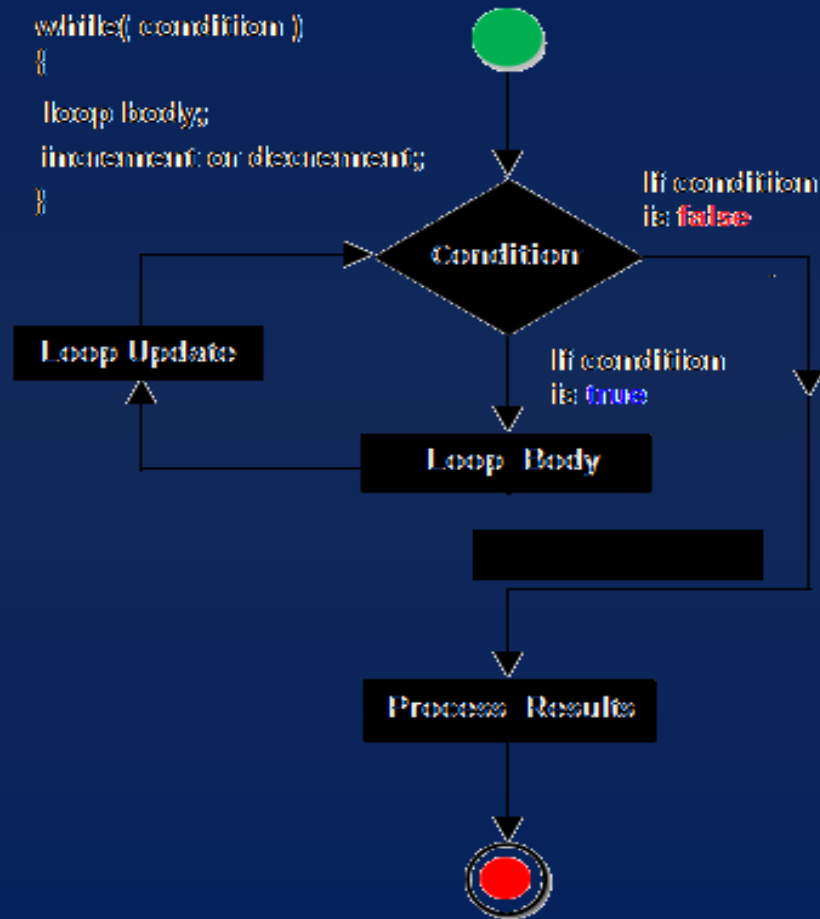
```
for(i=2;i<=50;i++);
```



1. Create a program to create a table of given number.
2. Write a program to print cube of numbers 1 to 5.
3. Create a program to print the factorial of a number. Factorial is $1*2*3*...*n$.
4. Write a program to accept a number from the user and display its divisors.
Ex: 12 = 1, 2, 3, 4, 6, 12
5. Write a program which accept a number (n) and find the sum from 1 to n.
6. Write a program to input any 10 integers number and find largest number.
7. Create a program that will display the elements of Fibonacci series between 1 and 100. eg. 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89



While loop



The while loop is entry controlled loop. The while loop keeps on executing the block of statement till the specified test condition evaluates to true. When the test condition becomes false, the program control is transferred out of the loop and passes to the statement after the body of the loop.

Syntax:

While(condition)

{

Statement 1;

Statement 2;

.....

Statement n;

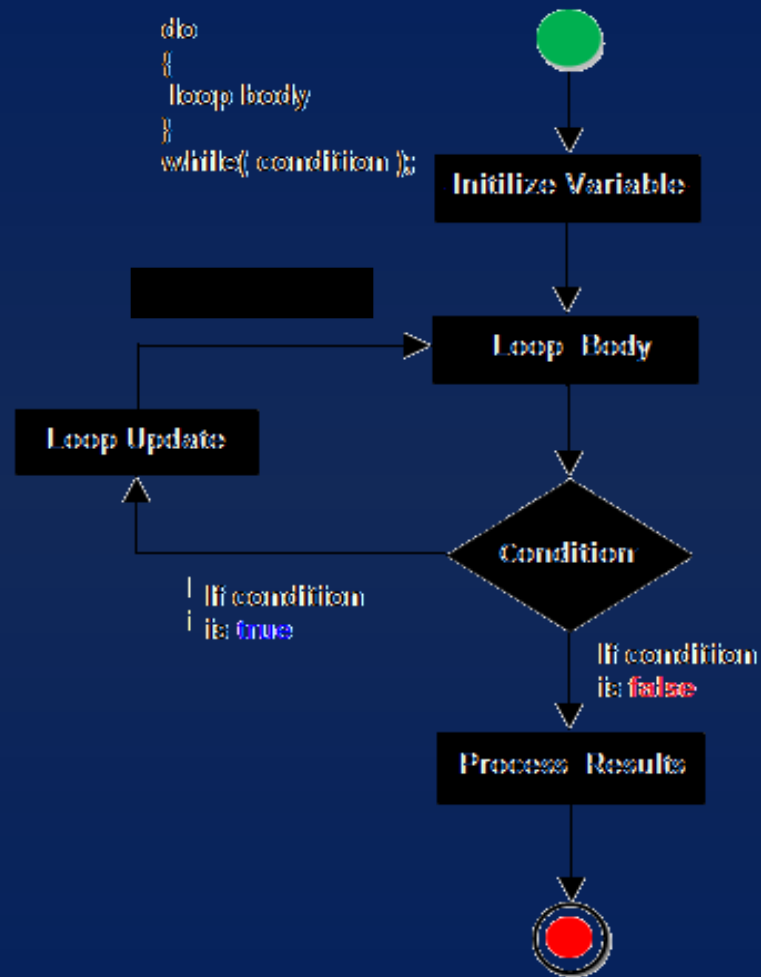
}



1. Write a program that will find the **reverse** of a given number.
2. Write a program which accept a number from user and **count the number of digits**.
3. Write a program that will find the **Sum of digits** of a given Number.
4. Write a program which accept a integer number from user and check, number is **Palindrome** or Not. Eg 121 ,444,100001 etc.
5. Write a program which accept a 3 digit integer number and check number is **Armstrong** or not. Ex 153, 370,371,407.
6. Write a program input any integer number and finds its largest and smallest digit. Ex. 92584 the min=2 and max=9
7. Write a Program which accept a number from user and check whether it is a **Disarium Number** or not. Ex. 135 is a DISARIUM ($1^1+3^2+5^3 = 135$), some other **DISARIUM** are 89, 175, 518 etc.)



do-while



The do while loop is an **exit controlled loop**, since the body of loop is executed first and then the test condition is evaluated. If it evaluates to false, then the loop is terminated, otherwise it is repeated. This means do while loop always executes **at least once**, even if the condition is evaluated to false.

Syntax:

```

do
{
    statement 1;
    statement 1;
    .....
    statement 1;
}while(condition);
  
```



- Write a program which print sum of negative numbers, sum of positive odd numbers and positive even numbers from a list of numbers entered by the user. The list terminates when the user enters a Zero.
- Write a program which accept integers number from user until the sum of all given number is less than 100.
-



Jumping Statement

The jump statement are used to transfer the program control unconditionally to some other location. Java provides three types of jump statements:

- Break statement
- Continue statement
- Return statement



- By using this jumping statement, we can terminate the further execution of the program and transfer the control to the end of any immediate loop or switch body.
- To do all this we have to specify a break jumping statements whenever we want to terminate from the loop.
- **Syntax:** break;
- **NOTE:** This jumping statements always used with the control structure like switch case, while, do while, for loop etc.

NOTE: As break jumping statements ends/terminate loop of one level . so it is referred to use return or goto jumping statements , during more deeply nested loops.



Continue Statements.

- By using this jumping statement, we can terminate the further execution of the program and transfer the control to the beginning of any immediate loop.
- To do all this we have to specify a continue jumping statements whenever we want to terminate any particular condition and restart/continue our execution.

Syntax: continue;

- **NOTE:** This jumping statements always used with the looping structure while, do while, for loop .



Return Statement

- return statement is used to transfer program's control from called function to calling function, it's secondary task is to carry value from called function to calling function.
- If function's return type is void, you can use return only to return program's control to the calling function, and if there is a return type you will have to return same type of value to the calling function.

Syntax:

return;

Or

return (value)





Methods/Function

- A function is a block containing a number of statements, which perform a specific and well defined task(s). Every function has a unique name which is used to identify it in a program. Once a task is packaged in a method, it can be called from anywhere in a program through other statements.

Advantages of function:

- Function reduce the **complexity** of program by dividing it into smaller modules, thus making them easier to read and debug.
- Function enhance **reusability** of code and can be called from anywhere in a program and any number of times.
- Functions are useful in hiding the implementation details.
- It is easier to edit a program, which is written using methods. While working on one function, one can focus on just that part of the program by debugging, constructing and making it perfect.



➤ Function can be broadly classified into two categories:

➤ **Built-in function or library functions**

These are inbuilt function, which are grouped together and stored in a standard java library. You can make use of these functions to get a predefined output, instead of writing your own code.

➤ **User defined Function :**

These function are designed by the user to carry out some specific tasks. These name of the method is given by the user according to his/her requirement; hence, it is called a user defined function.



- A method must be defined before it is used anywhere in the program. The general form of method definition consist of two parts.
 - Method Header/prototype
 - Body of method

Method Header

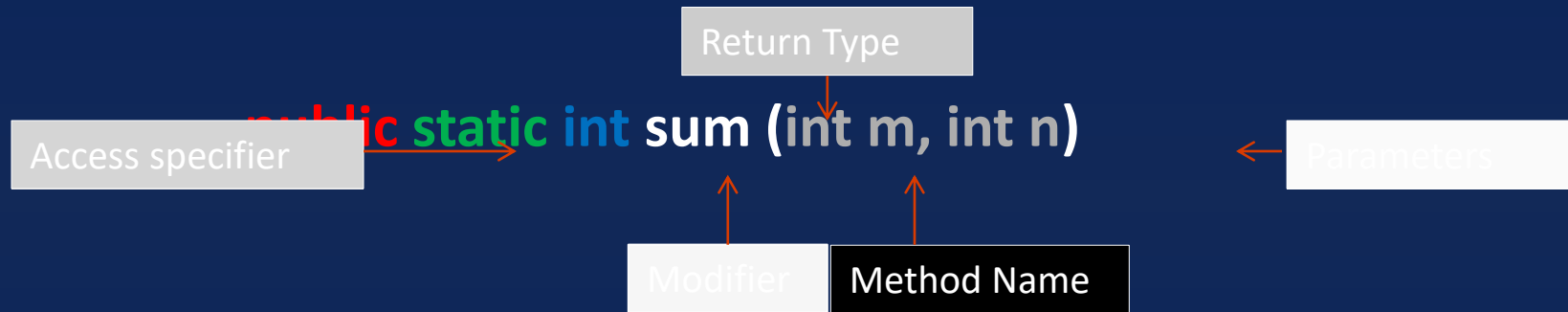
Access Specifier Modifier Return Type Method Name (arg1,arg2,arg3)

```
{  
  ---  
  ---  
  ---  
}
```

Body of Method



- A method header/prototype is the first line of method definition that tells the compiler about the method name, type of value returned by the method, and the types of arguments to be passed, if the function is expected to return some result, then the function prototype reveals the data type of the result.



- **Method with no argument and no return value**
- **Method with argument and no return value**
- **Method with argument and with return value**
- **Method with no argument and with return value**



- We can pass two type of arguments to a method
- We can pass any primitive data type argument to a java method, like Boolean, char, byte, char, int, long etc. This method of passing arguments is known as passing arguments by value.
- We can also pass reference data types, like object or array.
- A method can be called in two different ways:
 - **Call by Value or Pass by Value**
 - **Call by Reference or Pass by Reference**



- When you pass parameters to a method by value, the method copies the value of actual parameter into the formal parameter. It creates its own copy of argument values and then uses in the method.



- In this technique, actual arguments are passed by reference rather than passing by values. When function call is made, the reference of actual parameters is copied into formal parameters. Further this reference is used to access the values stored in a function. Since the function receive the location of the original value, the same value is used for all operations. If any changes ate made in the formal parameters, the original value automatically gets modified. This happen because the program has only one copy of the value.



Method in java classes are used to perform certain task based on the logic of the program. Java objects also store information or data in specific variables. Java methods in the course of their execution may or may not change the values in the variables of the objects created from the respective java class. On the basis of their functionality, the methods can be classified as pure methods and impure methods.

➤ **PURE(ACCESSOR/GETTER) Methods**

Pure methods do not change the original state of an object. They simply access that values that are passed to them as arguments. The function generally return a value to its caller method. The return value is either a primitive or a new object created inside the method. In a nutshell, we can say that a method does not change the original state of an object or simply return the initial state of the object is called pure method.



➤ **IMPURE(MUTATOR/PUTTER) Methods**

- Impure method change the initial state of an object, every time the are called. It means that the values of actual parameters get modified or change or changed depending upon the current state of the object on which the method operates. Impure methods may/may not return some result.



- Java Language facilitates the designing of the number of functions with the same name, which are, however, used for different purposes. They can have same name but their method signature must be different. Such functions are called **overloaded function** and the process of constructing and executing them is referred to as **Function Overloading**.
- When an overloaded method is invoked (called), the java compiler compares the numbers of parameters and the data types of the arguments with the parameters of the different versions of the method. The version which matches in both respects. i.e. the numbers and types of the parameters, get invoked. Thus, overloaded method must differ in the type and number of their parameters. When java receives a call to an overloaded method, its simply execute the version of the method whose **parameters match the arguments** used in the call.



Qus 1: Create a program to demonstrate function Overloading. The program must have three different versions of the function, calculateArea(). This function will calculate the area of circle, rectangle and square on the basis of type and number of arguments passed.

Qus 2: Design a class to overload a function num_cal() as follows:

- void num_cal(int num, char ch) with one integer arguments and one character arguments. It computes the square of an integer if choice ch is 's' otherwise, computes the cube.
- void num_cal(int a, int b, char ch) with tow integer arguments and one character argument. It compute the product of integer arguments if ch is 'p' else adds the integer.
- void num_cal(String str1,String str2) with tow String arguments and prints the two String are equal or not.



- Static method are also called class method because these methods can be called without creating an object of that class. A method is made static by using the keyword, 'static' in the method declaration. We can declare both functions and variables as static. The most common example of a static function is the main().
- As you seen that generally, the main() method of java program is declared with the keyword, 'static', so that main() can be executed by the interpreter without creating an object from the class.



Class as User Defined Data Type

Introduction

- Class is the fundamental base of all java programming. It is the logical construct upon which the entire programming code is built. It has the capability to define the state and behavior of its objects. The class implements the object-oriented programming(OOP) concepts in java. If we want wish to implement any OOP concept in a java program, then it must be encapsulated within a class.
- The most important feature about a class is that it defines a new data type. Once defined, this new type can be used to create objects of that type. Hence, we can say that a class is a template for an object, and an object is an instance of a class.



- A class determines the functionality of its objects. A class is considered as a blueprint or template, using which the object of the class are created. Hence, defining a class is a very important step in the implementation of object-Oriented programming.

Syntax:

```
<Access Specifier> <Modifier> <Class Name>
{
    Data Member 1
    Data Member 2
    .....
    Data Member n
    Member method-1()
    {
        .....
        .....
    }
}
```



- As you know, a class provides the blueprint for object creation. Whenever we create a new data type. We can use this data type to create new object that class and assign it to the variable.
- **Declaration** : declaration tells the compiler that you will be using a variable whose data type is class.
- **Instantiation** : this step uses the 'new' keyword that creates the object. The 'new' keyword allocates memory space for the object.
- **Initialization** : The 'new' keyword is followed by a call to a constructor, which initializes the attributes of the new object.

Syntax:

Class Name Object Name=new Constructor Name();



Accessing Variables and Function using objects

- We can access the instance variable and function by using the object of a class. To access any instance variable or function, we use the **dot operator (.)**.

Syntax: object.variableName

Ex: obj.amount=5000;



- An access specifier restricts the space of a function or members of a class. The sole purpose of an access specifier is to decide as to which entity can be accessed from where. The java language supports four access levels for variables and method: private, protected, public and default. While defining the class members, the desired access specifier is prefixed with its definition. In most cases, we either use the 'public' or the 'private' access specifier. When no access specifier is mentioned, then by default class members have 'default' access – known as friendly or package access. A package is a group of logically related classes.



- **Public** : The class members that are specified as public can be accessed directly by all classes, which are inside and outside the package.
- **Private** : The class members that are specified as private, can be accessed only within the scope of a class.
- **Protected** : protected members of a class are considered as being public to a subclass of a class, but private to all the other classes outside the current package.
- **Default** : The members with a default access can be accessed by all the sub classes and non-sub classes that are in the same package. However, these members cannot be accessed by class lying outside the package even if it is a sub class.



Access Specifier Table

91

Access Specifier	Within Class	Same Package Subclass	Same Package Non Subclass	Different Package Subclass	Different Package Non Subclass
private	Yes	No	No	No	No
public	Yes	Yes	Yes	Yes	Yes
protected	Yes	Yes	Yes	Yes	No
default	Yes	Yes	Yes	No	No

fomax

- Sometimes a method will need to refer to the object that invoked it. To Allow this java defines the 'this' keyword. 'this' can be used to indicate the current object. Some programmers prefer to always use 'this' when referring to member variable of the object whose method the reference appears. Doing so makes the intent of the code explicit and reduce errors based on name sharing.



- Sometimes, it is required to convert primitive data types into objects. For this purpose, wrapper class is used..
- Whenever any primitive data types are required as an object, wrapper class wraps the data type and gives it the appearance of an object. Wrapper class represents the primitive data type as an instance of its corresponding class.
- The Primitive wrapper class and their corresponding primitive data types are :

Data Type	Wrapper Class	Data Type	Wrapper Class
byte	Byte	short	Short
int	Integer	long	Long
char	Character	double	Double
float	Float	boolean	Boolean

- All method of wrapper classes are static
- Constructor of wrapper classes are not written like other user defined classes.
- Once we assign a value to a wrapper class instance, it cannot be changed.
- All wrapper class presents in java.lang package which is automatically imported in program.



The **Integer** class works as a wrapper class for the **int** data type. **Integer** class objects can be obtained from numeric values or string that contain valid whole numbers. **Integer** class has its own function, which can be called by using the following syntax:

Integer.Function_Name(Arguments);

Since all the function of a wrapper class are static in nature, hence we do not need an instance to call them.

Ex: **Integer.parseInt(str)**

➤ Some important functions of integer class are:

Function	Usage
<code>Ineteger.parseInt(String)</code>	<code>int n=lnetegr.parseInt("34");</code>
<code>Integer.valueOf(String)</code>	<code>int n=lnetegr.valueOf("34");</code>
<code>Inetegr.toString(int)</code>	<code>String str=lnetegr.toString(56);</code>

Character is a wrapper class for char data type. This class also has its own functions, which are mainly used to manipulate the character value. Character class can be called using the following syntax:

Character.function_name(Arguments);

Ex: Chatacter.toUpperCase('d');

Some important function of Character Class are:

Function	usage	Description
Character.isLetter(char)	boolean c=Character.isLetter('a')	Checks whether a character is a letter or not
Character.isDigit(char)	boolean c=Character.isDigit(3);	Checks whether a character is digit or not
Character.isletterOrDigit(char)	boolean c= Character.isLetterOrDigit('k');	Checks whether the character is either letter or digit

Character Wrapper Class (cont.)

97

Function	Usage	Description
Character.isLowerCase(char)	Boolean c =Character.isLowerCase('A') ;	Checks whether the character in lower case or not.
Character.isUpperCase(char)	Boolean c =Character.isUpperCase('A'));	Checks whether the character in upper case or not.
Character.isWhitespace(char)	Boolean c =Character.isWhitespace(' ');	Checks whether the character is whitespace or not.
Character.toUpperCase(char)	Char c =Character.toUpperCase('a') ;	To convert a character in upper case
Character.toLowerCase(char)	Char c =Character.toLowerCase('A'));	To convert a character in lower case



Wrapper Class	Function	Usage
Long	Long.parseLong(String)	long n=Long.parseLong("34")
Long	Long.valueOf(String)	long n=Long.valueOf("34")
Float	Float.parseFloat(String)	float f=Float.parseFloat("12.20")
Float	Float.valueOf(String)	float f=Float.valueOf("12.20")
Float	Float.toString(float)	String str=Float.toString(12.20);
Double	Double.parseDouble(String)	double d=Double.parseDouble("12.35");
Double	Double.valueOf(String)	double d=Double.valueOf("12.35");
Double	Double.toString(double)	String str=Double.toString(12.35339);

Define a class '**Discount**' having the following description:

Data members/instance variables:

int cost : to store the price of article

String name : to store name of customer

double dc : to store the discount

double amt : to store the amount to be paid

Member Methods:

void input() : store the cost of article, name of the customer

void calc() : calculate the 20% discount and amount to be paid.

void display() : display the name of the customer, cost, discount, amount to be paid



Lab assignment 2

10
0

Define a class '**CarParking**' with the following specifications:

Data members/instance variables:

car_num : to store the car number

cust_phone : to store the phone number of customer

cust_name : to store the name of customer

days : to store the number of days car will park

amount : to store the total rent to be paid

Member Method:

void input() : to input and store details of the car and customer

void compute() :to calculate total rent to be paid

Consider the following

Days	Rent
First 5 days	Rs. 500 per day
Next 5 days	Rs. 400 per day
After 10 days	Rs. 200 per day



Design a class '**Magic**' with the following specifications:

Data members/instance variables:

n : Store the number

Member function/methods:

void setNum(int nn) : to assign parameter value to the number, n=nn

int sumOfDigits(int n) : to return the sum of digit of a number.

void isMagic() : checks if the given number is a magic number or not by calling the function sumOfDigits(int n) and display appropriate message.

*A **magic number** is a number in which the eventual sum of the number is equal to 1.*

Ex : $172=1+7+2=10 \Rightarrow 10=1+0=1$ then 172 is a magic number.



Lab Assignment 4

10
2

Design a class '**Happy**' to check if a given number is a happy number or Not. Some of the members of class are given below:

Data members/instance Variable:

n : to store a number

Member Function/Methods:

void getnum(int nn) : to assign the parameter value(nn) to the n.

int sum_sq_digits(int x) : returns the sum of square of the digits of the number x .

void isHappy() : checks if the given number is happy number by calling the function sum_sq_digit(int) and display an appropriate message.

*A **Happy Number** is a number in which the eventual sum of the square of the digits of the number is equal to 1.*

Ex1: $28 = (2^2) + (8^2) = 4 + 64 = 68 \Rightarrow 68 = (6^2) + (8^2) = 36 + 64 = 100 \Rightarrow$
 $100 = (1^2) + (0^2) + (0^2) = 1 + 0 + 0 = 1$ hence 28 is a happy Number

Ex2: $12 = (1^2) + (2^2) = 1 + 4 = 5$ hence 12 is not Happy Number



Constructors

- Constructor are very important part of java. The word constructor in general signifies constructing something finite. Constructor comes into play when objects are created from class. Whenever objects are created, some initialization need to be done for the object. The java language provides flexibility to initialize all the data members of an object automatically whenever objects are created. This automatic initialization is done by using constructors.



- A constructor is a method that has no return type (not even void)
- The name of constructor is same as that of the class name in which it resides.
- A constructor is automatically called on object creation.
- You can have more than one constructor in your class, as long as the argument lists are different. Having more than one constructor in a class means you have overloaded constructors.
- If a constructor is not defined, then the compiler provides a default constructor, which takes no arguments and performs no special actions or initialization.
- Constructor may include parameters of various type. When the constructor is invoked using new operator, the type must match with those data type that are specified in the constructor definition.



Example:

10
6

```
Class Student
{
    int roll_no;
    int standard;
    public Student() // constructor
    {
        roll_no=1;
        standard=5;
    }
}
```



Difference between Constructor and Function

10
7

Constructor	Function
Class name and its constructor name must be same.	Functions name must be different from the class name.
Constructor create and initialize new object.	Function perform operations on existing object.
Constructor cannot be called directly, they are called implicitly when the keyword 'new' creates an object. Thus, they are called at the time of object creation.	Function can be called directly on an object that has already been created with 'new'. They are called when some specific task has been encountered.
If a constructor is not defined in a class, compiler supplies the default constructor i.e. constructor with Zero/No argument and empty body.	This is not the case with function.
Constructor have no return type not even void	Function should have a return type. If the function does not return any value, it should be void.
Execution of at least one of the constructor is a must.	All the function defined in a class may not execute.

max

- Constructor can be divided into two major categories:
 - Default Constructor (Non - Parameterized)
 - Parameterized Constructor



Default Constructor are also known as non-parameterized constructors. At the name suggests, these constructors have no parameters and hence do not accepts any arguments. If we do not define any constructor, java automatically provides a default constructor for the class. It can be used to initialize all the data members of the class with zero or null. When we define our own constructor, the default constructor is no longer used by the compiler.

Ex: Class MSWord

```
{  
    int font_size;  
    String font_color;  
    MSWord()  
    {  
        font_size=12;  
        font_color="black";  
    }  
}
```



A parameterized constructor always accepts arguments when an object of the class is created. A parameterized constructor uses its arguments to initialize the data members of the class for its new object. It makes the program more flexible when applied to a real life scenario since the program now has the capability of creating objects with different states.

Ex: Class MSWord

```
{  
    int font_size;  
    String font_color;  
    MSWord(int size, String color)  
    {  
        font_size=size;  
        font_color=color;  
    }  
}
```



- The concept of **Constructor Overloading** is very similar to **Function Overloading**. The process of creating more than one constructor with same name in a class is called Constructor Overloading.
- Just like the mandatory conditions of Function Overloading, the signature of all versions of the constructor should be different for Constructor Overloading. i.e. they should have different number and data types, or arrangement of parameters.
- While creating an object of the class, some arguments are passed during the constructor call. The java compiler checks the number and the data types of the arguments and calls that particular version of constructor whose number and data types of parameters match with the passed arguments



Example of Constructor Overloading

11
2

```
Class Pattern
{
    int row;
    char symbol;
    Pattern()
    {
        row=5;
        symbol='*';
    }

    Pattern(int r)
    {
        row=r;
        symbol='*';
    }

    Pattern(int r, char c)
    {
        row=r;
        symbol=c;
    }

    void print()
    {
        for(int i=1;i<=row;i++)
        {
            for(int j=1;j<=i;j++)
            {
                System.out.print(symbol);
            }
            System.out.println();
        }
    }

    public static void main(String arrgs[])
    {
        Pattern p1=new Pattern();
        p1.print();
        Pattern p2=new Pattern(8);
        p2.print();
        Pattern p3=new Pattern(4,'@');
        p3.print();
    }
}
```



A copy constructor is used to create another object, which is a copy of the object that it takes as a parameter. However, the newly created copy is actually independent of the original object. The copied is located at a completely different address in the memory away from the original.

A copy constructor is a constructor with one parameter of the same type as the class. It should be designed so the object is create is intuitively an exact, but a completely independent copy of its parameter.

Example:

```
int length, breadth;  
Rectangle(int l, int b)  
{  
    length=l;  
    breadth=b;  
}
```

```
Rectangle(Rectangle r)  
{  
    length=r.length;  
    breadth=r.breadth;  
}  
  
Public static void main(String arr[])  
{  
    Rectangle r1=new Rectangle(5,10);  
    Rectangle r2=new Rectangle(r1);  
}
```



Design a class **Prod** to perform the product of digits of a number input by user. The class details are as:

Class Name : Prod

Data members/instance Variables:

num	:	integer to store a number.
product	:	to store the product of digits.

Member function/method:

Prod()	:	constructor to assign 0 to num.
Prod(int nx)	:	parametrized constructor to assign nx to num .
void getProduct()	:	to find and store the product of all digits of num.
void display()	:	to print the number and product of all the digits.



The coordinate of a **point** on a two dimensional plane can be represented by $P(x,y)$; an 'x' co-ordinate and a 'y' co-ordinate.

Co-ordinate of the midpoint between points 2,3 and 4,7 can be calculated as :
 $((2+4)/2, (3+7)/2) = (3,5)$ To find the solution of above problem a class **Point** is declared as Follows:

Class : Point

Data members/Instance variables:

double x,y : to store x,y co-ordinates.

Member functions/Method

Point() : constructor to initialize the $x=0, y=0$

void readPoint() : to read the coordinate x and y of point.

void displaypoint() : to print x,y coordinate with suitable heading.

Point midpoint(Point A, Point B) : to calculate the co-ordinate of the mid point between the two given points p1,p2 and returns them.



Packages

A java Package can be considered as a collection of java classes that are grouped on the basis of their functionality.

Advantage of Package

- Package group related classes, so that they can be managed effectively.
- Whenever a class is required, it can be used by importing the parent package.
- We can declare multiple classes with same name, but in different package.
- Encapsulation can be implemented by using package.

Type of Package

package are divided into major categories:

- Java Library Package
- User Defined Package



These package are already present with the java language. All the built-in classes are grouped together in packages, like **java.lang**, **java.io**, **java.util**, etc. These package (except java.lang), are imported by the programmers as per their requirement. The **java.lang** package is the only package that gets invoked automatically for all java programs. This package provides the basic functionality of java language. The classes, such as **String**, **StringBuffer**, **Math**, **System** and **all Wrapper classes** are part of the java.lang package.



These package are defined by the user and stored with unique name. These package are imported in java programs by using the import command.

Creating A Package:

To create a package in java, we write the keyword **package**, followed by the name of the new package. This statement should be the first statement in the program file.

Syntax: <Package> <Package Name>

Ex: package shapes;

The Import Statement : The biggest advantage of a package is that once we define and store a package; it can be imported anytime in a project by using **import** statement in the beginning of the project. The import statement followed by the name of a package which is to be imported.

Syntax: **import** PackageName.*;



Exception Handling

An exception is an unexpected abnormal condition (not necessarily errors, but loosely referred to as error), that occur during the execution of a program. These errors are not syntax errors or logical errors caused by the programmer. These error occur either due to wrong/unexpected inputs or due a wrong reference to unavailable locations. When an exception occurs in a program, technically we say that an exception has been raised or thrown.

Hence, exception are the errors that occur during the execution od a program. The technique to handle such error is known as **Exception Handling**.

Exception can be classified into three categories:

- **Checked Exception:** These are exception that cannot be foreseen by programmer. A file is supposed to opened, and can not be found – a checked exception occurs.
- **Runtime Exception:** These are exceptions that can foreseen by the programmer and can be avoided (or handled)
- **Errors:** These cannot be handled at all, as the programmer cannot do anything at all.



Example crash the system.

- The try, catch, and finally are the java keywords that are used to handle exception.



Array

Introduction

12
4

An array is a collection of similar data types. Array is a container object that hold values of homogenous type. It is also known as static data structure because size of an array must be specified at the time of its declaration. Index of array starts from zero to size-1.

Array Declaration:

```
datatype identifier[ ]=new datatype[size];
```

Example:

```
int[ ] arr = new int[10]; //10 is the size of array.
```



Initialization of Array

12
5

datatype identifier[]={value1,value2,...};

Example:

```
int[ ] arr = {10,20,30,40,50};
```

Accessing array element:

```
array_name[index number];
```

Example:

```
arr[0]=56;
```

```
System.out.println(arr[0]);
```



String Function & Description

[char charAt\(int index\)](#)

Returns the character at the specified index.

[int compareTo\(String anotherString\)](#)

Compares two strings lexicographically.

[int compareToIgnoreCase\(String str\)](#)

Compares two strings lexicographically, ignoring case differences.

[String concat\(String str\)](#)

Concatenates the specified string to the end of this string.

[boolean endsWith\(String suffix\)](#)

Tests if this string ends with the specified suffix.

[boolean equals\(String anString\)](#)

Compares this string to the specified string.

[boolean equalsIgnoreCase\(String anotherString\)](#)

Compares this String to another String, ignoring case considerations.

Function

int indexOf(char ch)

Returns the index within this string of the first occurrence of the specified character.

int lastIndexOf(int ch)

Returns the index within this string of the last occurrence of the specified character.

int length()

Returns the length of this string.

String replace(char oldChar, char newChar)

Returns a new string resulting from replacing all occurrences of oldChar in this string with newChar.

boolean startsWith(String prefix)

Tests if this string starts with the specified prefix.

String substring(int beginIndex, int endIndex) Returns a new string that is a substring of this string.

Function
<u>String toLowerCase()</u> Converts all of the characters in this String to lower case using the rules of the default locale.
<u>String toString()</u> This object (which is already a string!) is itself returned.
<u>String toUpperCase()</u> Converts all of the characters in this String to upper case using the rules of the default locale.
<u>String trim()</u> Returns a copy of the string, with leading and trailing whitespace omitted.
<u>static String valueOf(primitive data type x)</u> Returns the string representation of the passed data type argument.
<u>char[] toCharArray()</u> Converts this string to a new character array.

